

Entangled
Minds

Amaravati Quantum Valley Hackathon 2025



- **Problem Statement ID** - AQVH919
- **Problem Statement Title**- Logistics: Fleet Optimization
- **Theme**- Quantum-Driven Dynamic Fleet Routing
- **PS Category**- Software
- **Team ID**- U0037@202305103
- **Team Name** - Entangled Minds



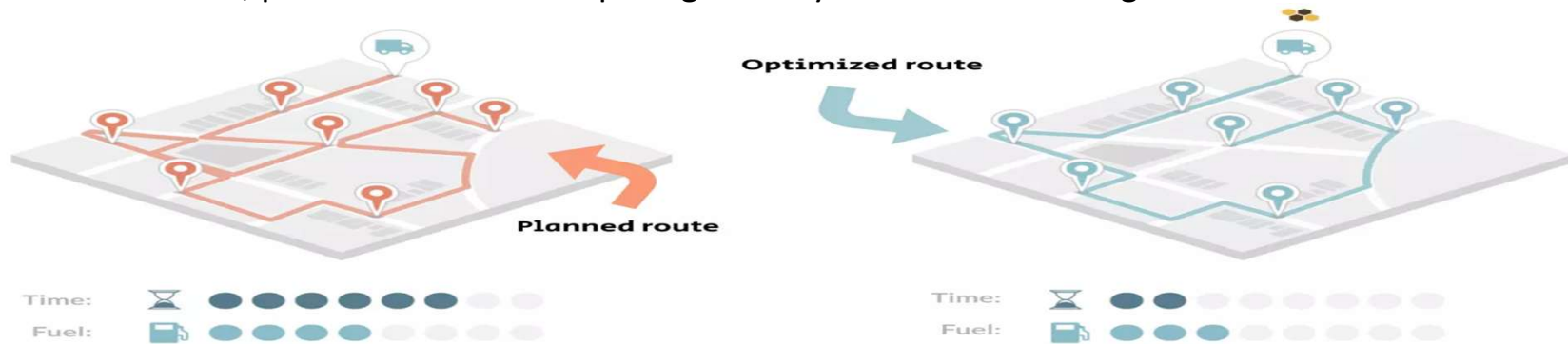
Quantum-Driven Vehicle Path Optimization in Logistics Networks

Proposed Solution

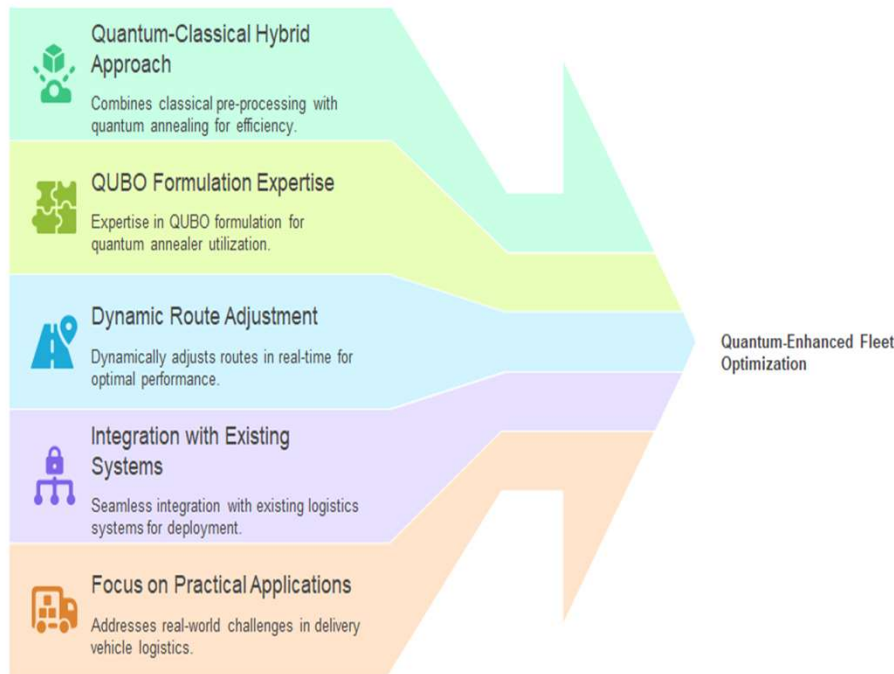
Transform traditional vehicle routing by adopting a hybrid quantum-classical framework for dynamic, real-time optimization. Moving beyond static plans, the system continuously adapts to traffic, orders, and vehicle status for optimal efficiency.

How the Quantum-Enhanced Routing Solution Addresses the Vehicle Routing Problem

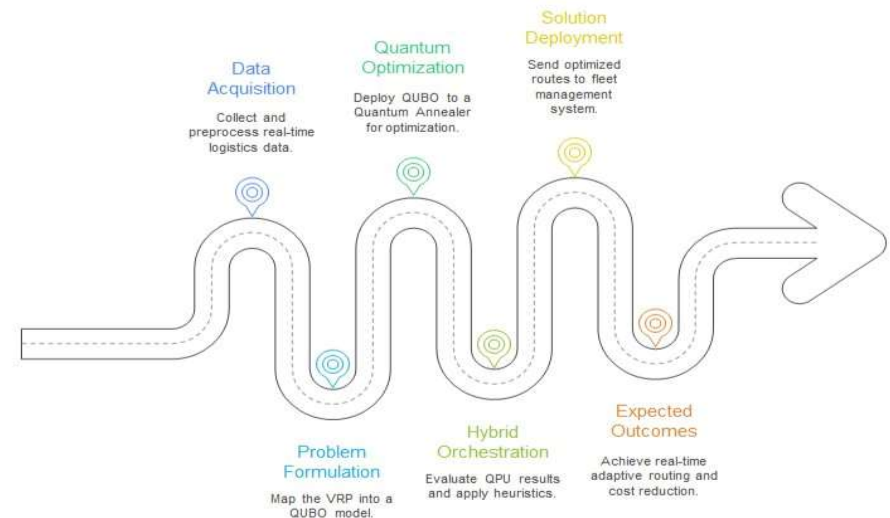
- Combines quantum and classical techniques for faster route computation.
- Allows real-time updates by integrating live traffic, orders, and vehicle data.
- Utilizes quantum annealing to quickly optimize routes amid disruptions.
- Provides a scalable, predictive framework paving the way for autonomous logistic



Quantum-Driven Logistics Innovation



TECHNICAL APPROACH

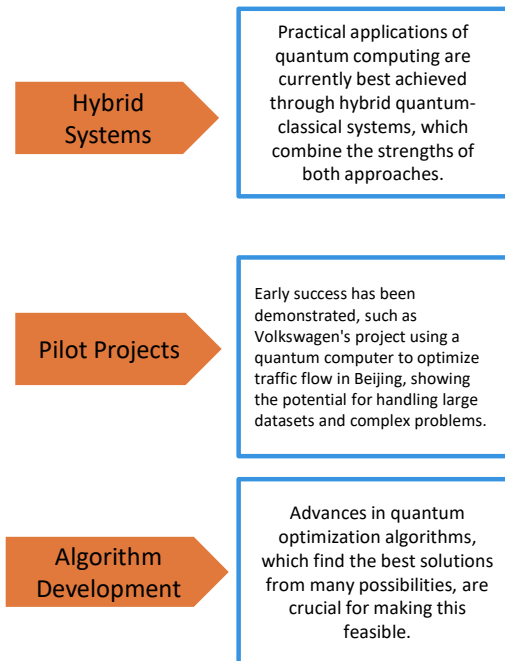


Languages: Python

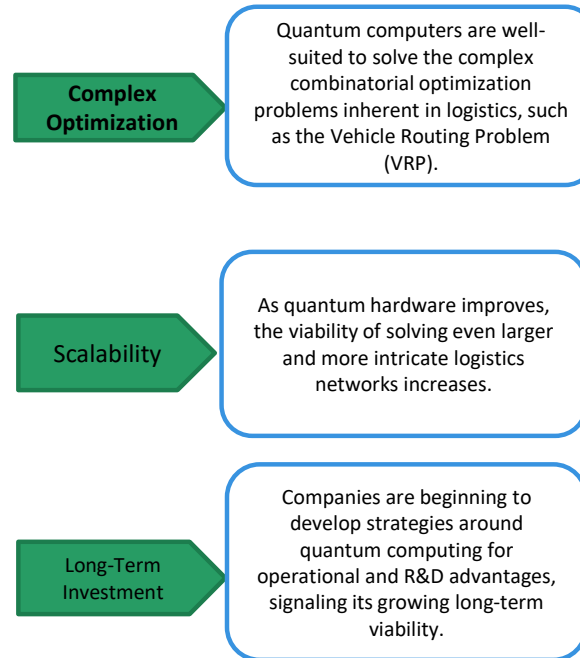
Frameworks: Quantum computing SDK: D-Wave Ocean SDK, Classical ML, Tensor Flow, scikit Learn, Hybrid Work flows

Hardware/Cloud Platforms: Quantum Annealer, Classical Servers

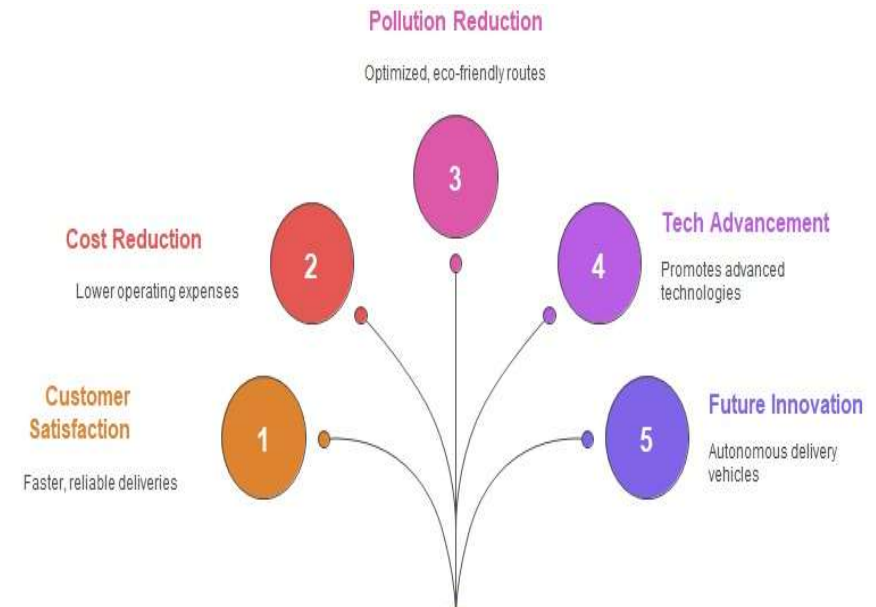
Feasibility



Viability



Impacts & Benefits



Based on Quantum annealer simulation & QUBO

`pip install dimod dwave-networkx`

```
print("QUBO Dictionary (sample of first 10 terms):")
print(list(qubo.items())[:10])
print("Number of QUBO variables (requires {} qubits):".format(len(qubo)))

# 3. Solve the QUBO using a Simulated Annealer
# dimod's SimulatedAnnealingSampler emulates the behavior of a quantum annealer
sampler = dimod.SimulatedAnnealingSampler()

# Run the sampler, num_reads is the number of times to run the annealing process.
sampleset = sampler.sample_qubo(qubo, num_reads=1000)

# 4. Interpret and Print the Results
# Get the best sample (lowest energy solution)
best_sample = sampleset.first_sample
energy = sampleset.first_energy

print("Best sample found: ", best_sample)
print("Energy (cost) of solution: ", energy)

# 5. Decode the solution back into a route
# The sample uses binary variables x_ij, position in tour
route = [None] * len(G) # Create an empty list for the route

# Iterate through all the variables in the best sample
for node, value in best_sample.items():
    # The variable name is a tuple (city, time_step)
    if value == 1:
        city, time_step = node
        route[time_step] = city

# The route is a list of cities in the order they are visited
print("Optimized Route Order: ", route)
# Since it's a cycle, we start and end at the same city. Let's complete the cycle.
route.append(route[0])
print("Complete Cycle Route: ", route)

# 6. Calculate the total distance of the found route
total_distance = 0
for i in range(len(route)-1):
    city_i = route[i]
    city_j = route[i+1]
    total_distance += G[city_i][city_j]["weight"]

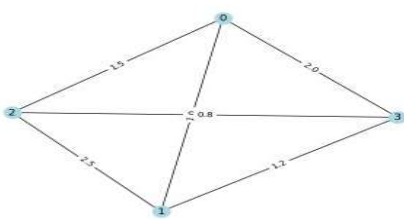
print("Total Distance of the Route: ", total_distance)

# 7. (Optional) Plot the optimized route
plt.figure()
nx.draw(G, pos, with_labels=True, node_color='lightblue')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

# Draw the path of the route
route_edges = [(route[i], route[i+1]) for i in range(len(route)-1)]
nx.draw_networkx_edges(G, pos, edgelist=route_edges, edge_color='red', width=2)
nx.draw_networkx_nodes(G, pos, nodelist=route, node_color='red')

plt.title("Optimized TSP Route")
plt.show()
```

City Network Map



```
QUBO Dictionary (sample of first 10 terms):
{((0, 0), (0, 0)): -12.0, ((0, 0), (0, 1)): 12.0, ((0, 0), (0, 2)): 12.0, ((0, 0), (0, 3)): 12.0, ((0, 0), (0, 4)): -12.0, ((0, 1), (0, 1)): 12.0, ((0, 1), (0, 2)): 12.0, ((0, 1), (0, 3)): 12.0, ((0, 1), (0, 4)): -12.0, ((0, 2), (0, 2)): 12.0, ((0, 2), (0, 3)): 12.0, ((0, 2), (0, 4)): -12.0, ((0, 3), (0, 3)): 12.0, ((0, 3), (0, 4)): -12.0, ((0, 4), (0, 4)): -12.0}

Number of QUBO variables (requires 136 qubits):
```

```
QUBO Dictionary (sample of first 10 terms):
{((0, 0), (0, 0)): -12.0, ((0, 0), (0, 1)): 12.0, ((0, 0), (0, 2)): 12.0, ((0, 0), (0, 3)): 12.0, ((0, 0), (0, 4)): -12.0, ((0, 1), (0, 1)): 12.0, ((0, 1), (0, 2)): 12.0, ((0, 1), (0, 3)): 12.0, ((0, 1), (0, 4)): -12.0, ((0, 2), (0, 2)): 12.0, ((0, 2), (0, 3)): 12.0, ((0, 2), (0, 4)): -12.0, ((0, 3), (0, 3)): 12.0, ((0, 3), (0, 4)): -12.0, ((0, 4), (0, 4)): -12.0}

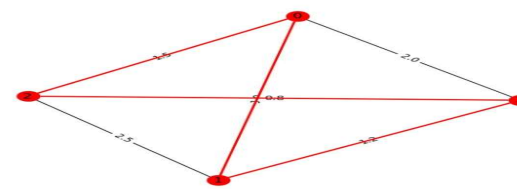
Number of QUBO variables (requires 136 qubits):

Best sample found: ((0, 0): np.int8(0), (0, 1): np.int8(0), (0, 2): np.int8(1), (0, 3): np.int8(0), (0, 4): np.int8(0), (1, 0): np.int8(0), (1, 1): np.int8(1), (1, 2): np.int8(0), (1, 3): np.int8(0), (1, 4): np.int8(0), (2, 0): np.int8(0), (2, 1): np.int8(0), (2, 2): np.int8(0), (2, 3): np.int8(1), (2, 4): np.int8(0), (3, 0): np.int8(1), (3, 1): np.int8(1), (3, 2): np.int8(1), (3, 3): np.int8(0), (3, 4): np.int8(0), (4, 0): np.int8(0), (4, 1): np.int8(0), (4, 2): np.int8(0), (4, 3): np.int8(1), (4, 4): np.int8(1))

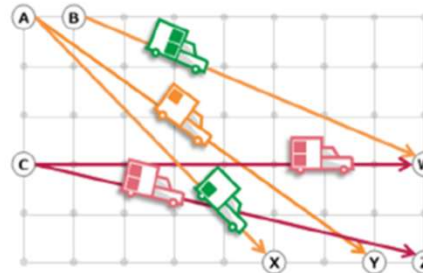
Energy (cost) of solution: -43.5

Optimized Route Order: [3, 4, 0, 2]
Complete Cycle Route: [3, 4, 0, 2, 3]
Total Distance of the Route: 4.5
```

Optimized TSP Route

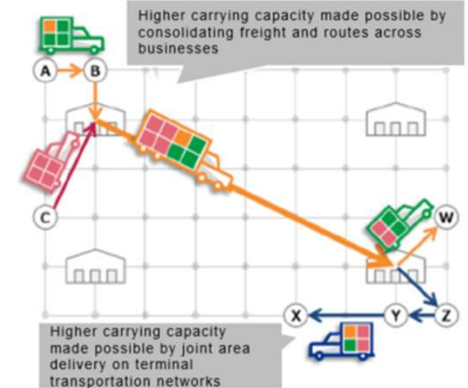


Individual businesses' exclusive networks

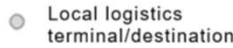


Different colors represent different logistics operators.

Cross-business or cross-sector network



Open and shared hub



Local logistics terminal/destination

Key Research Papers/Articles:

- “Quantum algorithms for vehicle routing problems”
- “Quantum annealing for solving vehicle routing problems”
- “The Vehicle Routing Problem”
- “Quantum Annealing for Vehicle Routing and Fleet Management”
<https://ieeexplore.ieee.org/document/8756014>
- “Hybrid Quantum-Classical Optimization for Fleet Routing”